

05_final_forward_model

April 27, 2026

1 05 - Final Forward Prediction Model

This notebook builds the final forward prediction pipeline for the Boom Challenge.

The objective is to train one final prediction system that returns all six ejecta outcomes:

- P80
- fines_frac
- oversize_frac
- R95
- R50_fines
- R50_oversize

The final system is target-specific: each target can use the model and feature set that performed best during controlled validation.

Current selected strategy based on previous notebooks:

Target	Model	Feature set
P80	ExtraTrees	target-family-specific
fines_frac	ExtraTrees	target-family-specific
oversize_frac	CatBoost	extended-plus-regimes
R95	ExtraTrees	extended-plus-regimes
R50_fines	ExtraTrees	extended-plus-regimes
R50_oversize	ExtraTrees	extended-plus-regimes

This notebook also includes optional linear baselines with Ridge, Lasso, and ElasticNet for comparison, but the final submission is generated using the target-specific tree/boosting pipeline.

1.1 1. Imports and Project Paths

```
[4]: from pathlib import Path
import json
import warnings
import joblib

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso, ElasticNet

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 180)

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.
    ↪ cwd()
FORWARD_DIR = PROJECT_ROOT / "data" / "raw" / "forward_prediction"
INVERSE_DIR = PROJECT_ROOT / "data" / "raw" / "inverse_design"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
SUBMISSIONS_DIR = OUTPUTS_DIR / "submissions"
MODELS_DIR = OUTPUTS_DIR / "models"
REPORTS_DIR = PROJECT_ROOT / "reports"

for path in [SUBMISSIONS_DIR, MODELS_DIR, REPORTS_DIR]:
    path.mkdir(parents=True, exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Forward data directory:", FORWARD_DIR)
print("Inverse design directory:", INVERSE_DIR)
print("Submissions directory:", SUBMISSIONS_DIR)
print("Models directory:", MODELS_DIR)

```

```

Project root: /home/alouiyaz/projects/boom-challenge-ejecta-prediction
Forward data directory: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/data/raw/forward_prediction
Inverse design directory: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/data/raw/inverse_design
Submissions directory: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions
Models directory: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/models

```

1.2 2. Load Data and Constraints

The constraints.json file is not required to train the forward prediction model, but it is useful for evaluating whether the model behaves reasonably near the inverse-design target zone.

```

[5]: input_cols = [
    "energy", "angle_rad", "coupling", "strength",
    "porosity", "gravity", "atmosphere", "shape_factor"

```

```

]

target_cols = [
    "P80", "fines_frac", "oversize_frac",
    "R95", "R50_fines", "R50_oversize"
]

fragmentation_targets = ["P80", "fines_frac", "oversize_frac"]
distance_targets = ["R95", "R50_fines", "R50_oversize"]

raw_train = pd.read_csv(FORWARD_DIR / "train.csv")[input_cols]
raw_test = pd.read_csv(FORWARD_DIR / "test.csv")[input_cols]
y = pd.read_csv(FORWARD_DIR / "train_labels.csv")[target_cols]
submission_template = pd.read_csv(FORWARD_DIR / "prediction_submission_template.
    ↪csv")

constraints_path = INVERSE_DIR / "constraints.json"
if not constraints_path.exists():
    constraints_path = PROJECT_ROOT / "constraints.json"

if constraints_path.exists():
    with open(constraints_path, "r") as f:
        constraints_data = json.load(f)
        output_constraints = constraints_data["constraints"]
        input_bounds = constraints_data["input_bounds"]
else:
    output_constraints = {"p80_min": 96.0, "p80_max": 101.0, "r95_max": 175.0}
    input_bounds = {}

print("Raw train:", raw_train.shape)
print("Raw test:", raw_test.shape)
print("Targets:", y.shape)
print("Constraints:")
display(pd.DataFrame([output_constraints]))

display(raw_train.head())
display(y.head())

```

Raw train: (2930, 8)

Raw test: (492, 8)

Targets: (2930, 6)

Constraints:

	p80_min	p80_max	r95_max
0	96.0	101.0	175.0

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	↪
↪shape_factor								

0	3.826405	0.818303	0.861258	1.305809	0.337215	3.71	0.781263	0.
	↪784028							
1	2.828754	1.193036	0.561245	3.494501	0.058029	1.62	0.136205	0.
	↪922737							
2	3.068907	0.605872	0.948860	1.366386	0.315632	3.71	0.774704	0.
	↪954922							
3	2.700574	1.073708	0.713705	3.599419	0.033062	1.62	0.144204	0.
	↪932911							
4	3.484022	0.863568	1.237205	1.996742	0.278207	9.81	0.414620	1.
	↪260855							

	P80	fines_frac	oversize_frac	R95	R50_fines	R50_oversize
0	76.972350	0.184728	0.016671	198.938699	175.527939	76.235779
1	269.057465	0.000622	0.916734	239.268477	447.157838	141.894047
2	104.070923	0.070343	0.094438	192.986417	189.286407	84.235774
3	257.618403	0.001026	0.880122	289.289693	500.000028	169.866473
4	111.717167	0.058576	0.136166	94.229304	97.614864	42.928393

1.3 3. Physics-Inspired Feature Engineering

These engineered variables are proxy features. They are not exact physical equations, because the dataset does not include projectile mass, projectile velocity, diameter, density, explicit material class, or impact location.

They help the model learn:

- transferred energy,
- angle decomposition,
- material resistance and fragility,
- gravity-scaled range,
- atmosphere and drag effects,
- discrete regimes observed during EDA.

```
[6]: def add_physics_features(X: pd.DataFrame) -> pd.DataFrame:
    X = X.copy()
    eps = 1e-9

    # 1. Energy transfer features
    X["effective_energy"] = X["energy"] * X["coupling"]
    X["log_energy"] = np.log1p(X["energy"])
    X["log_effective_energy"] = np.log1p(X["effective_energy"])

    # 2. Angle decomposition
    X["sin_angle"] = np.sin(X["angle_rad"])
    X["cos_angle"] = np.cos(X["angle_rad"])
    X["tan_angle"] = np.tan(X["angle_rad"])
    X["horizontal_energy"] = X["effective_energy"] * X["cos_angle"]
    X["vertical_energy"] = X["effective_energy"] * X["sin_angle"]
```

```

X["vertical_horizontal_ratio"] = X["vertical_energy"] / (
↪(X["horizontal_energy"] + eps)

# 3. Material / fragmentation proxies
X["energy_per_strength"] = X["energy"] / (X["strength"] + eps)
X["effective_energy_per_strength"] = X["effective_energy"] / (X["strength"] +
↪eps)
X["material_resistance_index"] = X["strength"] * (1 - X["porosity"])
X["fragmentation_index"] = X["effective_energy"] * X["porosity"] / (
↪(X["strength"] + eps)
X["coupling_porosity"] = X["coupling"] * X["porosity"]
X["coupling_atmosphere"] = X["coupling"] * X["atmosphere"]
X["porosity_strength"] = X["porosity"] * X["strength"]

# 4. Gravity and range proxies
X["energy_per_gravity"] = X["energy"] / (X["gravity"] + eps)
X["effective_energy_per_gravity"] = X["effective_energy"] / (X["gravity"] +
↪eps)
X["horizontal_energy_per_gravity"] = X["horizontal_energy"] / (X["gravity"] +
↪eps)
X["vertical_energy_per_gravity"] = X["vertical_energy"] / (X["gravity"] +
↪eps)

# 5. Atmosphere and drag proxies
X["drag_proxy"] = X["atmosphere"] * X["shape_factor"]
X["drag_per_gravity"] = X["drag_proxy"] / (X["gravity"] + eps)
X["atmosphere_shape_energy"] = X["atmosphere"] * X["shape_factor"] *
↪X["effective_energy"]
X["atmosphere_per_gravity"] = X["atmosphere"] / (X["gravity"] + eps)

# 6. Pi-like scaling proxies inspired by dimensional analysis
X["pi_gravity_proxy"] = (X["gravity"] * X["coupling"]) / (X["energy"] + eps)
X["pi_strength_proxy"] = X["strength"] / (X["gravity"] * X["coupling"] +
↪eps)
X["pi_atmosphere_proxy"] = X["atmosphere"] / (X["gravity"] * X["coupling"] +
↪eps)

# 7. Regime indicators observed during EDA
X["porosity_regime"] = (X["porosity"] > 0.15).astype(int)
X["strength_regime"] = (X["strength"] > 2.6).astype(int)
X["angle_regime"] = (X["angle_rad"] > 0.95).astype(int)
X["atm_regime"] = (X["atmosphere"] > 0.30).astype(int)
X["regime_combo"] = (
    X["porosity_regime"] * 8
    + X["strength_regime"] * 4
    + X["angle_regime"] * 2

```

```

        + X["atm_regime"]
    )

    # 8. Additional cross-regime proxies
    X["scaled_energy"] = X["effective_energy"] / (X["strength"] * np.
↪sqrt(X["gravity"])) + eps)
    X["fragility"] = X["porosity"] / (X["strength"] + eps)
    X["range_proxy"] = (X["effective_energy"] * (X["cos_angle"] ** 2)) / ↪
↪(X["gravity"] * X["strength"] + eps)
    X["energy_sin_angle"] = X["energy"] * X["sin_angle"]
    X["momentum_proxy"] = X["effective_energy"] * X["sin_angle"]

    # Risk-controlled atmosphere ratio
    X["coupling_per_atm_clipped"] = X["coupling"] / (X["atmosphere"] + 1e-3)
    X["log_coupling_per_atm"] = np.log1p(X["coupling_per_atm_clipped"])
    X["retention_factor"] = X["atmosphere"] * X["drag_proxy"] / (X["energy"] + ↪
↪eps)

    return X

X_fe_train = add_physics_features(raw_train)
X_fe_test = add_physics_features(raw_test)

print("X_fe_train:", X_fe_train.shape)
print("X_fe_test:", X_fe_test.shape)

def check_modeling_matrix(X: pd.DataFrame, name: str):
    numeric = X.select_dtypes(include=[np.number])

    print(f"\n{name}")
    print("Shape:", X.shape)
    print("Missing:", int(X.isna().sum().sum()))
    print("Infinite:", int(np.isinf(numeric).sum().sum()))
    print("Non-numeric:", [c for c in X.columns if c not in numeric.columns])

check_modeling_matrix(X_fe_train, "X_fe_train")
check_modeling_matrix(X_fe_test, "X_fe_test")

```

X_fe_train: (2930, 48)

X_fe_test: (492, 48)

X_fe_train

Shape: (2930, 48)

Missing: 0

Infinite: 0

Non-numeric: []

```
X_fe_test
Shape: (492, 48)
Missing: 0
Infinite: 0
Non-numeric: []
```

1.4 4. Feature Sets

The final model uses different feature sets depending on the target family.

- Fragmentation targets use material, energy-to-strength and regime-related features.
- Distance targets use gravity, range, drag and extended regime-related features.

```
[7]: raw_features = input_cols.copy()

core_physics_features = raw_features + [
    "effective_energy", "log_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "tan_angle",
    "horizontal_energy", "vertical_energy", "vertical_horizontal_ratio",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",
    ↪ "atmosphere_per_gravity",
]

extended_physics_features = core_physics_features + [
    "pi_gravity_proxy", "pi_strength_proxy", "pi_atmosphere_proxy",
    "scaled_energy", "fragility", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "coupling_per_atm_clipped", "log_coupling_per_atm", "retention_factor",
]

extended_plus_regimes_features = extended_physics_features + [
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪ "regime_combo"
]

fragmentation_features = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "drag_proxy", "atmosphere_shape_energy",
    "pi_strength_proxy", "pi_atmosphere_proxy", "scaled_energy", "fragility",
```

```

        "energy_sin_angle", "momentum_proxy",
        "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
        ↪ "regime_combo",
    ]

distance_features = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",
    ↪ "atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_atmosphere_proxy", "scaled_energy", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪ "regime_combo",
]

def clean_feature_list(features, X):
    seen = set()
    out = []
    for f in features:
        if f in X.columns and f not in seen:
            out.append(f)
            seen.add(f)
    return out

raw_features = clean_feature_list(raw_features, X_fe_train)
core_physics_features = clean_feature_list(core_physics_features, X_fe_train)
extended_physics_features = clean_feature_list(extended_physics_features,
    ↪ X_fe_train)
extended_plus_regimes_features =
    ↪ clean_feature_list(extended_plus_regimes_features, X_fe_train)
fragmentation_features = clean_feature_list(fragmentation_features, X_fe_train)
distance_features = clean_feature_list(distance_features, X_fe_train)
full_features = X_fe_train.columns.tolist()

feature_sets = {
    "raw": raw_features,
    "core_physics": core_physics_features,
    "extended_physics": extended_physics_features,
    "extended_plus_regimes": extended_plus_regimes_features,
    "fragmentation": fragmentation_features,
    "distance": distance_features,
    "full_features": full_features,
}

```



```

def get_features_for_feature_set(feature_set_name: str, target: str):
    if feature_set_name == "target_family_specific":
        return fragmentation_features if target in fragmentation_targets else distance_features
    return feature_sets[feature_set_name]

feature_summary = pd.DataFrame({
    "feature_set": list(feature_sets.keys()) + ["target_family_specific"],
    "n_features": [len(v) for v in feature_sets.values()] + [
        len(fragmentation_features) + len(distance_features)
    ],
})
display(feature_summary)

```

	feature_set	n_features
0	raw	8
1	core_physics	32
2	extended_physics	43
3	extended_plus_regimes	48
4	fragmentation	34
5	distance	33
6	full_features	48
7	target_family_specific target-dependent	

1.5 5. Model Builders

The final pipeline uses the best model per target selected from the previous controlled validation notebook.

If CatBoost is not installed, the notebook automatically falls back to ExtraTrees for `oversize_frac`.

```

[8]: def build_extratrees(random_state=42):
    return ExtraTreesRegressor(
        n_estimators=800,
        max_features="sqrt",
        min_samples_leaf=2,
        random_state=random_state,
        n_jobs=-1,
    )

try:
    from catboost import CatBoostRegressor

    def build_catboost(random_state=42):
        return CatBoostRegressor(
            iterations=1200,
            learning_rate=0.03,
            depth=6,

```

```

        l2_leaf_reg=5.0,
        loss_function="RMSE",
        random_seed=random_state,
        verbose=False,
    )

    CATBOOST_AVAILABLE = True
except Exception as e:
    print("CatBoost unavailable. Falling back to ExtraTrees for all targets.")
    print(e)
    CATBOOST_AVAILABLE = False

model_builders = {
    "ExtraTrees": build_extratrees,
}
if CATBOOST_AVAILABLE:
    model_builders["CatBoost"] = build_catboost

print("Available final model builders:", list(model_builders.keys()))

```

Available final model builders: ['ExtraTrees', 'CatBoost']

1.6 6. Metrics and Constraint Diagnostics

```

[9]: def regression_metrics(y_true, y_pred, target_name=None):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    out = {"MAE": mae, "RMSE": rmse, "R2": r2}
    if target_name is not None:
        out["normalized_MAE"] = mae / (y[target_name].std() + 1e-9)
    return out

def clip_predictions(preds, target):
    preds = np.asarray(preds).copy()
    if target in ["fines_frac", "oversize_frac"]:
        return np.clip(preds, 0, 1)
    return np.clip(preds, 0, None)

p80_min = output_constraints["p80_min"]
p80_max = output_constraints["p80_max"]
r95_max = output_constraints["r95_max"]

def feasibility_mask(df_targets):

```

```

    return (df_targets["P80"].between(p80_min, p80_max)) & (df_targets["R95"]
↪ <= r95_max)

def near_feasible_mask(df_targets):
    return (df_targets["P80"].between(80, 120)) & (df_targets["R95"] <= 250)

def constraint_violation_score(df_pred):
    p80_low = np.maximum(p80_min - df_pred["P80"], 0)
    p80_high = np.maximum(df_pred["P80"] - p80_max, 0)
    r95_violation = np.maximum(df_pred["R95"] - r95_max, 0)
    return p80_low + p80_high + r95_violation

def custom_constraint_metrics(y_true_df, y_pred_df):
    true_feasible = feasibility_mask(y_true_df)
    pred_feasible = feasibility_mask(y_pred_df)
    near_zone = near_feasible_mask(y_true_df)

    tp = int((true_feasible & pred_feasible).sum())
    fp = int((~true_feasible & pred_feasible).sum())
    fn = int((true_feasible & ~pred_feasible).sum())

    precision = tp / (tp + fp + 1e-9)
    recall = tp / (tp + fn + 1e-9)
    f1 = 2 * precision * recall / (precision + recall + 1e-9)
    violation = constraint_violation_score(y_pred_df)

    metrics = {
        "n_true_feasible": int(true_feasible.sum()),
        "n_pred_feasible": int(pred_feasible.sum()),
        "true_positive": tp,
        "false_positive": fp,
        "false_negative": fn,
        "feasible_precision": precision,
        "feasible_recall": recall,
        "feasible_f1": f1,
        "mean_pred_constraint_violation": float(violation.mean()),
        "median_pred_constraint_violation": float(np.median(violation)),
        "near_zone_count": int(near_zone.sum()),
    }

    if near_zone.sum() > 0:
        metrics["near_zone_MAE_P80"] = mean_absolute_error(
            y_true_df.loc[near_zone, "P80"], y_pred_df.loc[near_zone, "P80"]
        )
        metrics["near_zone_MAE_R95"] = mean_absolute_error(
            y_true_df.loc[near_zone, "R95"], y_pred_df.loc[near_zone, "R95"]
        )

```

```

metrics["near_zone_R95_bias"] = float((
    y_pred_df.loc[near_zone, "R95"] - y_true_df.loc[near_zone, "R95"]
).mean())

return metrics

```

1.7 7. Final Target-Specific Pipeline

This class behaves like one final model, but internally it uses one specialized model per target.

```

[10]: class TargetSpecificForwardModel:
    def __init__(self, target_configs, feature_engineering_func,
        feature_selector, random_state=42):
        self.target_configs = target_configs
        self.feature_engineering_func = feature_engineering_func
        self.feature_selector = feature_selector
        self.random_state = random_state
        self.models_ = {}
        self.feature_columns_ = None

    def fit(self, X_raw: pd.DataFrame, y_df: pd.DataFrame):
        X_fe = self.feature_engineering_func(X_raw)
        self.feature_columns_ = X_fe.columns.tolist()

        for i, target in enumerate(y_df.columns):
            config = self.target_configs[target]
            model_name = config["model"]
            feature_set_name = config["feature_set"]

            model_builder = model_builders[model_name]
            features = self.feature_selector(feature_set_name, target)

            model = model_builder(random_state=self.random_state + i)
            model.fit(X_fe[features], y_df[target])

            self.models_[target] = {
                "model": model,
                "model_name": model_name,
                "feature_set": feature_set_name,
                "features": features,
            }

        return self

    def predict(self, X_raw: pd.DataFrame) -> pd.DataFrame:
        X_fe = self.feature_engineering_func(X_raw)
        preds = pd.DataFrame(index=X_raw.index)

```

```

    for target, info in self.models_.items():
        model = info["model"]
        features = info["features"]
        pred = model.predict(X_fe[features])
        preds[target] = clip_predictions(pred, target)

    return preds[target_cols]

def describe(self):
    rows = []
    for target, info in self.models_.items():
        rows.append({
            "target": target,
            "model": info["model_name"],
            "feature_set": info["feature_set"],
            "n_features": len(info["features"]),
        })
    return pd.DataFrame(rows)

```

1.8 8. Final Target Configuration

These selections come from the previous controlled validation notebook.

```

[11]: target_configs = {
    "P80": {
        "model": "ExtraTrees",
        "feature_set": "target_family_specific",
    },
    "fines_frac": {
        "model": "ExtraTrees",
        "feature_set": "target_family_specific",
    },
    "oversize_frac": {
        "model": "CatBoost" if CATBOOST_AVAILABLE else "ExtraTrees",
        "feature_set": "extended_plus_regimes" if CATBOOST_AVAILABLE else ↵
        "target_family_specific",
    },
    "R95": {
        "model": "ExtraTrees",
        "feature_set": "extended_plus_regimes",
    },
    "R50_fines": {
        "model": "ExtraTrees",
        "feature_set": "extended_plus_regimes",
    },
    "R50_oversize": {

```

```

        "model": "ExtraTrees",
        "feature_set": "extended_plus_regimes",
    },
}

pd.DataFrame([
    {"target": t, **cfg} for t, cfg in target_configs.items()
])

```

```

[11]:
      target      model      feature_set
0      P80  ExtraTrees  target_family_specific
1  fines_frac  ExtraTrees  target_family_specific
2  oversize_frac  CatBoost  extended_plus_regimes
3      R95  ExtraTrees  extended_plus_regimes
4  R50_fines  ExtraTrees  extended_plus_regimes
5  R50_oversize  ExtraTrees  extended_plus_regimes

```

1.9 9. Optional Cross-Validation Check of Final Pipeline

This section re-validates the final target-specific system with OOF predictions.

It is useful as a final sanity check before training on the full dataset and producing the submission.

```

[12]: def cross_validate_final_pipeline(X_raw, y_df, n_splits=5, random_state=42):
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    oof = pd.DataFrame(index=y_df.index, columns=target_cols, dtype=float)
    fold_rows = []

    for fold, (train_idx, valid_idx) in enumerate(kf.split(X_raw), start=1):
        X_tr = X_raw.iloc[train_idx].reset_index(drop=True)
        X_va = X_raw.iloc[valid_idx].reset_index(drop=True)
        y_tr = y_df.iloc[train_idx].reset_index(drop=True)
        y_va = y_df.iloc[valid_idx].reset_index(drop=True)

        model = TargetSpecificForwardModel(
            target_configs=target_configs,
            feature_engineering_func=add_physics_features,
            feature_selector=get_features_for_feature_set,
            random_state=random_state + fold,
        )
        model.fit(X_tr, y_tr)
        pred_va = model.predict(X_va)
        pred_va.index = valid_idx
        oof.loc[valid_idx, target_cols] = pred_va[target_cols]

    for target in target_cols:
        metrics = regression_metrics(y_df.iloc[valid_idx][target],
        ↪ pred_va[target], target_name=target)

```

```

        metrics.update({"fold": fold, "target": target})
        fold_rows.append(metrics)

    fold_df = pd.DataFrame(fold_rows)

    overall_rows = []
    for target in target_cols:
        metrics = regression_metrics(y_df[target], oof[target],
        ↪target_name=target)
        metrics.update({
            "fold": "OOF",
            "target": target,
            "model": target_configs[target]["model"],
            "feature_set": target_configs[target]["feature_set"],
        })
        overall_rows.append(metrics)

    overall_df = pd.DataFrame(overall_rows)
    constraint_df = pd.DataFrame([custom_constraint_metrics(y_df, oof)])

    return overall_df, fold_df, oof, constraint_df

RUN_FINAL_CV = True

if RUN_FINAL_CV:
    final_cv_results, final_fold_results, final_oof, final_constraint_metrics =
    ↪cross_validate_final_pipeline(
        raw_train,
        y,
        n_splits=5,
        random_state=42,
    )
    display(final_cv_results.sort_values("normalized_MAE"))
    display(final_constraint_metrics)
else:
    print("Skipping final CV check.")

```

	MAE	RMSE	R2	normalized_MAE	fold	target	
↪model							
2	0.026111	0.035923	0.990154	0.072114	OOF	oversize_frac	↪
↪CatBoost							
1	0.006492	0.014260	0.956777	0.094632	OOF	fines_frac	↪
↪ExtraTrees							
0	7.711179	10.257217	0.975659	0.117271	OOF	P80	↪
↪ExtraTrees							
3	40.568955	67.052700	0.920960	0.170070	OOF	R95	↪
↪ExtraTrees							

```

4  50.291834  78.432461  0.899051          0.203694  00F      R50_fines  ␣
↳ExtraTrees    extended_plus_regimes
5  22.588116  38.500797  0.873895          0.208306  00F      R50_oversize ␣
↳ExtraTrees    extended_plus_regimes

    n_true_feasible  n_pred_feasible  true_positive  false_positive ␣
↳false_negative  feasible_precision  feasible_recall  feasible_f1 ␣
↳mean_pred_constraint_violation  \
0          35          39          13          26          ␣
↳22          0.333333          0.371429          0.351351          199.
↳768269

    median_pred_constraint_violation  near_zone_count  near_zone_MAE_P80 ␣
↳near_zone_MAE_R95  near_zone_R95_bias
0          135.963758          372          4.571787          ␣
↳25.343042          14.178817

```

1.10 10. Optional Linear Baselines

Ridge, Lasso and ElasticNet are not expected to beat ExtraTrees on this nonlinear multi-regime problem.

However, they are useful as scientific baselines and help demonstrate that the problem is not purely linear.

This section can be skipped if execution time is important.

```

[13]: def build_linear_model(model_name):
        if model_name == "Ridge":
            return Pipeline([
                ("scaler", StandardScaler()),
                ("model", Ridge(alpha=1.0, random_state=42)),
            ])
        if model_name == "Lasso":
            return Pipeline([
                ("scaler", StandardScaler()),
                ("model", Lasso(alpha=0.001, max_iter=20000, random_state=42)),
            ])
        if model_name == "ElasticNet":
            return Pipeline([
                ("scaler", StandardScaler()),
                ("model", ElasticNet(alpha=0.001, l1_ratio=0.5, max_iter=20000,
↳random_state=42)),
            ])
        raise ValueError(model_name)

def cross_validate_linear_baselines(X_fe, y_df,
↳feature_set_name="extended_plus_regimes", n_splits=5):

```



```

    features = extended_plus_regimes_features if feature_set_name == "extended_plus_regimes" else full_features
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)
    rows = []

    for model_name in ["Ridge", "Lasso", "ElasticNet"]:
        for target in target_cols:
            oof = np.zeros(len(y_df))
            for fold, (train_idx, valid_idx) in enumerate(kf.split(X_fe), start=1):
                X_tr = X_fe.iloc[train_idx][features]
                X_va = X_fe.iloc[valid_idx][features]
                y_tr = y_df.iloc[train_idx][target]

                model = build_linear_model(model_name)
                model.fit(X_tr, y_tr)
                pred = clip_predictions(model.predict(X_va), target)
                oof[valid_idx] = pred

            metrics = regression_metrics(y_df[target], oof, target_name=target)
            metrics.update({"model": model_name, "target": target, "feature_set": feature_set_name})
            rows.append(metrics)

    return pd.DataFrame(rows)

RUN_LINEAR_BASELINES = False

if RUN_LINEAR_BASELINES:
    linear_baseline_results = cross_validate_linear_baselines(X_fe_train, y, feature_set_name="extended_plus_regimes", n_splits=5)
    display(linear_baseline_results.sort_values(["target", "normalized_MAE"]))
else:
    print("Linear baseline section skipped. Set RUN_LINEAR_BASELINES=True to run it.")

```

Linear baseline section skipped. Set RUN_LINEAR_BASELINES=True to run it.

1.11 11. Train Final Model on Full Training Data

```

[14]: final_model = TargetSpecificForwardModel(
    target_configs=target_configs,
    feature_engineering_func=add_physics_features,
    feature_selector=get_features_for_feature_set,
    random_state=42,
)

```

```
final_model.fit(raw_train, y)
final_model_description = final_model.describe()
display(final_model_description)
```

	target	model	feature_set	n_features
0	P80	ExtraTrees	target_family_specific	34
1	fines_frac	ExtraTrees	target_family_specific	34
2	oversize_frac	CatBoost	extended_plus_regimes	48
3	R95	ExtraTrees	extended_plus_regimes	48
4	R50_fines	ExtraTrees	extended_plus_regimes	48
5	R50_oversize	ExtraTrees	extended_plus_regimes	48

1.12 12. Predict Test Set and Generate Forward Submission

```
[15]: test_predictions = final_model.predict(raw_test)

display(test_predictions.head())
print(test_predictions.describe().T)

submission = pd.DataFrame({
    "scenario_id": np.arange(len(raw_test)),
})

for col in target_cols:
    submission[col] = test_predictions[col].values

# Ensure exact expected column order
submission = submission[[
    "scenario_id",
    "P80",
    "fines_frac",
    "oversize_frac",
    "R95",
    "R50_fines",
    "R50_oversize",
]]

submission_path = SUBMISSIONS_DIR /
    ↪ "prediction_submission_final_target_specific.csv"
submission.to_csv(submission_path, index=False)

print("Saved submission to:", submission_path)
display(submission.head())
display(submission.tail())
```

	P80	fines_frac	oversize_frac	R95	R50_fines	R50_oversize
0	122.051072	0.113360	0.174124	708.999379	684.516201	308.249491
1	112.594270	0.049793	0.156310	206.870162	222.623610	101.635447

2	151.683379	0.110091	0.173198	799.042923	773.467971	351.587836
3	159.635194	0.009869	0.561122	490.585081	570.468630	257.809107
4	139.734454	0.047011	0.220873	197.438058	239.681417	98.741241

	count	mean	std	min	25%	50%
75%	max					
P80	492.0	154.410581	29.205843	77.932694	131.014871	152.472220
172.902258	219.854089					
fines_frac	492.0	0.047445	0.051733	0.002168	0.007697	0.019887
0.079795	0.240072					
oversize_frac	492.0	0.432358	0.224458	0.045259	0.222882	0.416259
0.639375	0.801378					
R95	492.0	288.544250	230.479605	48.541924	110.308957	171.983077
463.871385	945.481917					
R50_fines	492.0	323.398563	226.841124	63.706866	142.964387	207.213529
560.527439	825.765013					
R50_oversize	492.0	142.488865	105.015312	27.498061	59.377240	88.196924
248.848512	406.201873					

Saved submission to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions/prediction_submission_final_target_specific.csv

scenario_id	P80	fines_frac	oversize_frac	R95	R50_fines	R50_oversize
0	122.051072	0.113360	0.174124	708.999379	684.516201	308.249491
1	112.594270	0.049793	0.156310	206.870162	222.623610	101.635447
2	151.683379	0.110091	0.173198	799.042923	773.467971	351.587836
3	159.635194	0.009869	0.561122	490.585081	570.468630	257.809107
4	139.734454	0.047011	0.220873	197.438058	239.681417	98.741241

scenario_id	P80	fines_frac	oversize_frac	R95	R50_fines	R50_oversize
487	155.110903	0.079211	0.223865	99.934974	106.953602	46.986966
488	135.888617	0.024405	0.357056	534.629955	607.101676	275.774256
489	163.379006	0.007736	0.568148	56.319157	69.499042	31.023591
490	184.626643	0.005017	0.722777	141.544935	198.415842	82.292775
491	198.642828	0.003228	0.727034	336.527582	473.763438	193.328917

1.13 13. Test Prediction Diagnostics

This section checks whether test predictions are physically plausible and identifies test scenarios predicted to satisfy the inverse-design constraints.

```
[16]: test_feasible = feasibility_mask(test_predictions)
print("Number of test scenarios predicted feasible:", int(test_feasible.sum()))

predicted_feasible_test = pd.concat(
    [pd.DataFrame({"scenario_id": np.arange(len(raw_test))}), raw_test,
    ↪test_predictions],
    axis=1,
).loc[test_feasible].copy()

if len(predicted_feasible_test) > 0:
    predicted_feasible_test["constraint_violation"] =
    ↪constraint_violation_score(predicted_feasible_test)
    display(predicted_feasible_test.sort_values(["constraint_violation", "R95",
    ↪"P80"]).head(20))
else:
    print("No test scenario is predicted to be feasible with the final forward
    ↪model.")

# Save diagnostics
predicted_feasible_path = REPORTS_DIR /
    ↪"predicted_feasible_test_scenarios_forward_model.csv"
predicted_feasible_test.to_csv(predicted_feasible_path, index=False)
print("Saved predicted feasible test scenarios to:", predicted_feasible_path)
```

Number of test scenarios predicted feasible: 2

	scenario_id	energy	angle_rad	coupling	strength	porosity	gravity	
	↪atmosphere	↪shape_factor		P80	↪fines_frac	↪oversize_frac		R95
	↪R50_fines	\						
156	156	4.428487	0.804516	1.461307	2.085320	0.158319	10.47	↪
	↪0.305087	1.046824	97.974234	0.133544		0.123533	104.475156	101.
	↪799706							
276	276	3.285417	1.018322	1.124264	0.986445	0.218214	7.03	↪
	↪0.302112	1.020648	97.284455	0.170943		0.074008	163.067787	159.
	↪174819							
		R50_oversize	constraint_violation					
156		43.542028		0.0				
276		68.806355		0.0				

Saved predicted feasible test scenarios to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports/predicted_feasible_test_scenarios_forward_model.csv

1.14 14. Save Final Model and Metadata

```
[17]: model_path = MODELS_DIR / "final_target_specific_forward_model.joblib"
      joblib.dump(final_model, model_path)

      metadata = {
          "target_configs": target_configs,
          "target_columns": target_cols,
          "input_columns": input_cols,
          "submission_path": str(submission_path),
          "model_path": str(model_path),
          "constraints": output_constraints,
      }

      metadata_path = MODELS_DIR / "final_target_specific_forward_model_metadata.json"
      with open(metadata_path, "w") as f:
          json.dump(metadata, f, indent=2)

      print("Saved model to:", model_path)
      print("Saved metadata to:", metadata_path)
```

Saved model to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/models/final_target_specific_forward_model.joblib
Saved metadata to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/models/final_target_specific_forward_model_metadata.json

1.15 15. Final Summary

This notebook creates the final forward prediction system.

Main outputs:

- outputs/submissions/prediction_submission_final_target_specific.csv
- outputs/models/final_target_specific_forward_model.joblib
- outputs/models/final_target_specific_forward_model_metadata.json
- reports/predicted_feasible_test_scenarios_forward_model.csv

Next step:

Create the inverse-design notebook. That notebook will generate candidate input scenarios inside the allowed bounds from `constraints.json`, predict their ejecta outcomes using an ensemble of models, filter candidates satisfying $96 \leq P80 \leq 101$ and $R95 \leq 175$, and select the final 20 design scenarios.